SLOTWISE / MERN PORTFOLIO SERIES

Your Complete MERN Roadmap

Models, APIs, booking logic,
screens, testing, and deployment.

PART 02 / BUILD ROADMAP

BUILD GOAL

Includes complete starter schemas for all four collections, route examples, a race-safe booking pattern, and checklists. Keep it focused: one provider and 30-minute appointments.

Follow this from a blank folder to a portfolio-ready demo.

6 phases | 4 Mongoose models | 1 deployable app

START HERE / PROJECT BOUNDARY

Before writing any code

Build **SlotWise**: a booking website for one small business (for example, a salon). A customer chooses a service and a 30-minute slot. An admin defines the services and weekly opening hours, then manages bookings.

> MVP CONTRACT
>
> **One provider.** Only one customer can be served at a time. **Every service is 30 minutes.** Slots are aligned to :00 and :30. **One admin.** Add that account using a seed script, not public registration. No payments, emails, or multiple businesses.

Recommended stack

LAYER	USE
Frontend	React + Vite, React Router, regular CSS or Tailwind
Backend	Node.js + Express with JavaScript ES modules
Database	MongoDB Atlas or local MongoDB + Mongoose
Authentication	Password hashing (bcrypt), JWT stored in an HttpOnly cookie
Date and time	Luxon for business timezone conversion
Tests	Vitest or Jest, Supertest, one browser flow with Playwright
Deployment	Frontend host + Node host + MongoDB Atlas

Definition of done

- ☐ Customer can register, log in, see services, book, view, and cancel.
- ☐ Admin can edit services and weekly hours, and view upcoming appointments.
- ☐ The API rejects bookings that are in the past, outside opening hours, or already taken.
- ☐ The app is deployed and works after a refresh on mobile and desktop.

SETUP / DAY ONE

Start the repo and run your API

Begin with one React client and one Express server. Keep them in separate folders, but inside the same GitHub repository. Use JavaScript ES modules consistently in the backend.

Starter terminal commands

Quick setup (example)

```
mkdir slotwise && cd slotwise
npm create vite@latest client -- --template react
mkdir server && cd server
npm init -y
npm install express mongoose dotenv cors cookie-parser bcryptjs jsonwebtoken luxon
npm install -D nodemon
# Set "type": "module" in server/package.json
cd ../client && npm install
npm install react-router-dom
```

Example server environment variables

server/.env - do not commit this file

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/slotwise
JWT_SECRET=replace_with_a_long_random_secret
BUSINESS_TZ=Asia/Kathmandu
NODE_ENV=development
```

Day-one sanity checks

- Start the React dev server, then start Express on its configured port.
- Create GET /api/health that responds with a small JSON object.
- Confirm that Express can connect to MongoDB; never print connection secrets in logs.

> **A HELPFUL GIT HABIT**
>
> Commit a **server/.env.example** containing variable names with placeholder values. Ignore the real **.env** in Git.

ROADMAP / WEEKS 1-2

Phase 1-2: Foundation and accounts

Week 1 - Project setup + database

- ☐ Create a GitHub repository with **client/** and **server/**.
- ☐ Set up Vite + React; install Express, Mongoose, dotenv, and cors.
- ☐ Connect Express to MongoDB and add a **GET /api/health** endpoint.
- ☐ Create the four models: User, Service, WorkingHours, Appointment.
- ☐ Seed 2-3 services and one week of opening hours.
- ☐ Check data with MongoDB Compass or the Atlas UI.

> **WEEK 1 CHECKPOINT**
>
> Start both apps locally. A browser request to **/api/health** succeeds, and the seeded services exist in MongoDB.

Week 2 - Auth + roles

- ☐ Implement **POST /api/auth/register** (always create customer role).
- ☐ Implement login/logout with password hashing and a secure cookie.
- ☐ Implement **GET /api/auth/me** and authentication middleware.
- ☐ Add an **admin-only** role middleware and seed the admin account.
- ☐ Create React login and registration pages with error messages.
- ☐ Protect the Admin routes on the server, not just with hidden UI.

> **WEEK 2 CHECKPOINT**
>
> A customer cannot call an admin-only API. The admin can log in, and an unknown user gets 401 on a protected route.

ROADMAP / WEEKS 3-4

Phase 3-4: Make booking work

Week 3 - Services, availability, booking API

- ☐ Build **GET /api/services** for active services.
- ☐ Admin builds create/update/disable service endpoints.
- ☐ Admin sets weekly opening hours, such as 09:00-17:00.
- ☐ Build **GET /api/slots?date=YYYY-MM-DD** to return free half-hours.
- ☐ Build **POST /api/appointments** with server-side slot checks.
- ☐ Add a unique partial index, and return **409 Conflict** on duplicate slots.
- ☐ Implement list own appointments and cancel own appointment.

> **WEEK 3 CHECKPOINT**
>
> Two simultaneous requests for 10:00 cannot both succeed. Cancelled slots can be booked again.

Week 4 - Customer-facing React pages

- ☐ Build service cards and a date picker (start with a simple date input).
- ☐ Fetch available slots each time the date changes.
- ☐ Create a selected-slot state and a confirmation screen.
- ☐ Handle loading, empty results, 401, and 409 responses clearly.
- ☐ Build the **My Appointments** page with Cancel action.
- ☐ Check layouts on a mobile-size screen.

> **WEEK 4 CHECKPOINT**
>
> A user can book from the browser, refresh, see the booking, cancel it, and rebook the same time.

ROADMAP / WEEKS 5-6

Phase 5-6: Admin, tests, and deployment

Week 5 - Admin dashboard and testing

- ☐ Add admin pages for services, weekly hours, and the appointment list.
- ☐ Filter appointments by date; show customer and service names.
- ☐ Test authentication, role checks, booking conflict, time validation, and cancellation.
- ☐ Add a confirmation modal before cancelling and friendly error messages.
- ☐ Add an empty-state illustration or text for no available appointments.

> **WEEK 5 CHECKPOINT**
>
> Every core flow works, and the important API error cases are tested.

Week 6 - Ship and explain

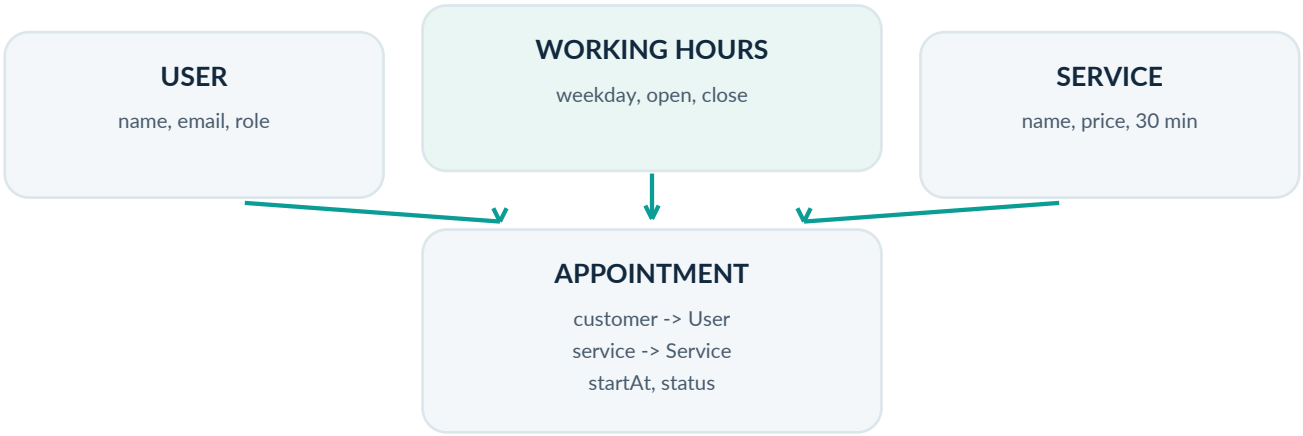
- ☐ Deploy the database, Express API, and React frontend.
- ☐ Use environment variables; never commit JWT secrets or database credentials.
- ☐ Configure cookie, CORS, HTTPS, and the frontend API URL correctly.
- ☐ Create demo customer and admin accounts (with safe test-only credentials).
- ☐ Write a README with screenshots, features, model relationships, and setup steps.
- ☐ Record a 60-90 second demo and link the live website in your resume.

> **WEEK 6 CHECKPOINT**
>
> An interviewer can open the live link, book a slot, see the booking in the admin dashboard, and understand the system from the README.

These are suggested weekly milestones. If a week takes longer, continue in order; do not add optional features before the core flow passes.

DATABASE / MODEL MAP

Design the four MongoDB models



Relationships to remember

- **User -> Appointment:** one customer can have many appointments; an appointment belongs to exactly one customer.
- **Service -> Appointment:** one service can appear in many appointments; keep service references by ObjectId.
- **WorkingHours -> Slots:** your server uses business hours to generate slots; slots are not a MongoDB collection.

Model conventions

- Add **timestamps: true** to track createdAt and updatedAt.
- Use **required**, **enum**, **min**, and meaningful defaults to validate fields.
- Store appointment start and end as **Date** (UTC instants), not strings like "10 AM".
- Treat role, status, and **blocksSlot** as server-owned fields; do not trust client-provided values.

> IMPORTANT TECHNICAL DISTINCTION
>
> A Mongoose **unique** option builds a MongoDB index; it is **not** a normal schema validator. Make sure indexes actually exist before you test concurrent bookings.

Suggested implementation folders

Backend and frontend organization

```
server/src/
  models/        User.js, Service.js, WorkingHours.js, Appointment.js
  routes/        auth.js, services.js, slots.js, appointments.js
  middleware/    auth.js, requireAdmin.js
  utils/         slots.js
client/src/
  pages/         Services.jsx, Book.jsx, MyAppointments.jsx
  pages/admin/   ServicesAdmin.jsx, HoursAdmin.jsx, Bookings.jsx
  components/    SlotPicker.jsx, ServiceCard.jsx
```

DATABASE / MODEL EXAMPLES 1-2

User and Service schemas

The samples below use JavaScript ES modules and Mongoose. Save each example in the matching **server/src/models/** file.

models/User.js

```
import { Schema, model } from 'mongoose';

const userSchema = new Schema({
  name: { type: String, required: true, trim: true },
  email: {
    type: String, required: true, unique: true,
    lowercase: true, trim: true
  },
  passwordHash: {
    type: String, required: true, select: false
  },
  role: {
    type: String, enum: ['customer', 'admin'],
    default: 'customer'
  }
}, { timestamps: true });

export default model('User', userSchema);
```

Why this shape: You store a password **hash**, never the plain password. The public register route must set role to customer even if the request body says admin.

models/Service.js

```
import { Schema, model } from 'mongoose';

const serviceSchema = new Schema({
  name: { type: String, required: true, trim: true },
  description: { type: String, default: '' },
  price: { type: Number, required: true, min: 0 },
  durationMinutes: {
    type: Number, enum: [30], default: 30
  },
  isActive: { type: Boolean, default: true }
}, { timestamps: true });

export default model('Service', serviceSchema);
```

Why exactly 30? Fixed duration makes the first version easier to build and protects the one-slot / one-appointment assumption. You can expand later.

DATABASE / MODEL EXAMPLES 3-4

WorkingHours and Appointment schemas

models/WorkingHours.js

```
import { Schema, model } from 'mongoose';

const timePattern = /^([01]\d|2[0-3]):[0-5]\d$/;
const hoursSchema = new Schema({
  weekday: {
    type: Number, required: true, unique: true,
    min: 0, max: 6 // 0 = Sunday, 6 = Saturday
  },
  openTime: { type: String, required: true, match: timePattern },
  closeTime: { type: String, required: true, match: timePattern },
  isClosed: { type: Boolean, default: false }
}, { timestamps: true });

export default model('WorkingHours', hoursSchema);
```

Validation: The admin API must additionally check that openTime is earlier than closeTime and both fit the 30-minute grid. A closed day can still keep default times.

models/Appointment.js

```
import { Schema, model } from 'mongoose';

const appointmentSchema = new Schema({
  customer: {
    type: Schema.Types.ObjectId, ref: 'User', required: true
  },
  service: {
    type: Schema.Types.ObjectId, ref: 'Service', required: true
  },
  startAt: { type: Date, required: true },
  endAt: { type: Date, required: true },
  status: {
    type: String, enum: ['confirmed', 'cancelled'],
    default: 'confirmed'
  },
  blocksSlot: { type: Boolean, default: true }
}, { timestamps: true });

appointmentSchema.index(
  { startAt: 1 },
  { unique: true, partialFilterExpression: { blocksSlot: true } }
);
export default model('Appointment', appointmentSchema);
```

The index enforces one active reservation per start time. It is correct here because **all bookings use the same aligned 30-minute slot** for one provider.

DATA SHAPE / REAL EXAMPLE

Example documents and time handling

Example: confirmed appointment

```
{
  "customer": "671faaa1111111111111111",
  "service": "671fbbb22222222222222222",
  "startAt": "2026-11-10T04:15:00.000Z",
  "endAt": "2026-11-10T04:45:00.000Z",
  "status": "confirmed",
  "blocksSlot": true
}
```

Example: weekly hours document

```
{
  "weekday": 2,
  "openTime": "09:00",
  "closeTime": "17:00",
  "isClosed": false
}
```

Three rules that prevent timezone bugs

- Pick one business timezone, such as **Asia/Kathmandu**, in backend configuration. Never rely on the server machine's local timezone.
- Generate local opening-hour slots using a timezone-aware library such as **Luxon**; convert each slot to UTC ISO before sending it over the API.
- Save startAt / endAt as MongoDB Dates. On the frontend, display the slot using the same business timezone.

> HOW TO READ THE EXAMPLE
>
> 10:00 AM on 10 November 2026 in Kathmandu (UTC+05:45) is **04:15 UTC**, so the stored start time is **2026-11-10T04:15:00.000Z**.

What happens on cancellation?

Illustrative atomic update

```
await Appointment.updateOne(
  { _id: appointmentId, customer: req.user.id,
    status: 'confirmed', startAt: { $gt: new Date() } },
  { $set: { status: 'cancelled', blocksSlot: false } }
);
```

Use the matched/modified count to decide whether cancellation succeeded. This prevents a user cancelling another customer's record.

BACKEND / REST API DESIGN

Endpoints to implement

METHOD	ROUTE	WHO / PURPOSE
POST	/api/auth/register	Public - create a customer
POST	/api/auth/login	Public - set session cookie
POST	/api/auth/logout	Signed-in - clear cookie
GET	/api/auth/me	Signed-in - current user
GET	/api/services	Public - active services
GET	/api/slots?date=YYYY-MM-DD	Public - free slots for date
POST	/api/appointments	Customer - confirm one slot
GET	/api/appointments/mine	Customer - own bookings
PATCH	/api/appointments/:id/cancel	Customer - cancel own booking
GET	/api/admin/appointments	Admin - all appointments
POST	/api/admin/services	Admin - create service
PATCH	/api/admin/services/:id	Admin - edit / disable service
PUT	/api/admin/hours/:weekday	Admin - update working hours

Simple request and response

POST /api/appointments

```
// Request body (user is read from the session)
{
  "serviceId": "671fbbb22222222222222222",
  "startAt": "2026-11-10T04:15:00.000Z"
}

// Success: 201 Created
{ "id": "671fccc33333333333333333", "status": "confirmed" }

// Slot taken: 409 Conflict
{ "message": "That slot was just booked. Pick another." }
```

Use consistent HTTP errors: 400 for bad input, 401 if not logged in, 403 for forbidden roles, 404 for missing records, and 409 for a slot collision.

THE HARD PART / SLOTS

How to generate available times

You do **not** store every empty slot in MongoDB. Calculate the day's possible slots from WorkingHours, then remove already-confirmed appointments.

Slot generation algorithm

01 Read the requested date (**YYYY-MM-DD**) and convert it to the business timezone.

02 Find the WorkingHours row for that weekday. If closed, return an empty array.

03 From opening time to closing time, create consecutive 30-minute intervals.

04 Skip times in the past. Query active bookings within that day using **blocksSlot: true**.

05 Remove booked UTC start times, then send the remaining starts as ISO strings.

Pseudo-code for GET /api/slots

```
const date = req.query.date; // validate YYYY-MM-DD
const localDay = parseBusinessDate(date, BUSINESS_TIMEZONE);
const hours = await WorkingHours.findOne({ weekday: localDay.weekday });
if (!hours || hours.isClosed) return res.json([]);

const starts = generateHalfHours(localDay, hours); // UTC Dates
const booked = await Appointment.find({
  startAt: { $gte: dayStartUtc, $lt: nextDayStartUtc },
  blocksSlot: true
}).select('startAt');

const taken = new Set(booked.map(a => a.startAt.toISOString()));
res.json(starts.filter(d => d > new Date() &&
  !taken.has(d.toISOString())).map(d => d.toISOString()));
```

DO NOT SKIP THIS DETAIL

The "free slots" response is only a preview. Before saving a booking, the server must **revalidate the chosen slot**. The unique database index handles concurrent requests at the final step.

For your single timezone, Luxon can convert a business-local start to a UTC instant with **DateTime.fromISO(..., { zone: TZ }).toUTC()**. Take care to use local weekdays when reading WorkingHours.

BOOKING VALIDATION / HELPER CODE

Example: validate one chosen slot

The booking API uses this helper to reject an invalid time before saving. This version assumes one provider, a 30-minute grid, same-day opening hours, and a fixed business timezone.

utils/slots.js - isOpenSlot() implementation

```
import { DateTime } from 'luxon';
import WorkingHours from '../models/WorkingHours.js';

const TZ = 'Asia/Kathmandu'; // single business timezone

export async function isOpenSlot(startAt) {
  if (!(startAt instanceof Date) ||
      Number.isNaN(startAt.getTime()) ||
      startAt <= new Date()) return false;

  const local = DateTime.fromJSDate(startAt, { zone: TZ });
  if (!local.isValid || local.minute % 30 !== 0 ||
      local.second !== 0 || local.millisecond !== 0) {
    return false;
  }

  const weekday = local.weekday % 7; // Luxon: Sunday=7
  const hours = await WorkingHours.findOne({ weekday });
  if (!hours || hours.isClosed) return false;

  const time = local.toFormat('HH:mm');
  const end = local.plus({ minutes: 30 }).toFormat('HH:mm');
  return time >= hours.openTime && end <= hours.closeTime;
}
```

Why each check exists

- **Future:** no booking a time that has already passed.
- **Alignment:** 10:00 and 10:30 are valid; 10:15 is not.
- **Business timezone:** use local weekday/open hours, not UTC weekday.
- **Availability:** this helper checks schedule rules. The unique database index separately blocks slots that someone else already reserved.

> **WEEK 3 IMPLEMENTATION TIP**
>
> Write unit tests for this helper with a closed day, a valid weekday, 10:15, a past time, and the last allowed slot. Then connect it to POST /api/appointments.

THE HARD PART / CONFIRMING

Example race-safe booking route

This is a practical route skeleton. It depends on an **isOpenSlot()** helper which checks local business hours, 30-minute alignment, and future time. You implement that helper alongside your slot generation logic.

routes/appointment.routes.js (core creation logic)

```
import { Router } from 'express';
import mongoose from 'mongoose';
import Appointment from '../models/Appointment.js';
import Service from '../models/Service.js';
import { requireAuth } from '../middleware/auth.js';
import { isOpenSlot } from '../utils/slots.js';

const router = Router();
router.post('/', requireAuth, async (req, res, next) => {
  try {
    const { serviceId, startAt } = req.body;
    if (!mongoose.isValidObjectId(serviceId) ||
        typeof startAt !== 'string' || !startAt.endsWith('Z')) {
      return res.status(400).json({ message: 'Invalid input' });
    }

    const service = await Service.findOne({
      _id: serviceId, isActive: true
    });
    if (!service) return res.sendStatus(404);

    const starts = new Date(startAt);
    if (Number.isNaN(starts.getTime()) ||
        !(await isOpenSlot(starts))) {
      return res.status(400).json({ message: 'Invalid slot' });
    }

    const booking = await Appointment.create({
      customer: req.user.id, service: service._id,
      startAt: starts,
      endAt: new Date(starts.getTime() + 30 * 60 * 1000),
      status: 'confirmed', blocksSlot: true
    });
    return res.status(201).json({ id: booking.id, status: booking.status });
  } catch (err) {
    if (err.code === 11000) {
      return res.status(409).json({ message: 'That slot was just booked' });
    }
    next(err);
  }
});
export default router;
```

Four things to verify

- Only a signed-in user reaches this route; **req.user.id** comes from your auth middleware.
- **isOpenSlot()** rejects anything outside business hours, in the past, or off the :00/:30 grid.
- The unique index has been created in MongoDB before booking tests run.
- The client never sends customerId, status, blocksSlot, or endAt as trusted values.

FRONTEND / SCREENS AND DATA

Build the React interface in this order

PAGE	CONTENT	API
Services	Service cards, name, price	GET /services
Book	Date input, slot buttons, confirm	GET /slots + POST /appointments
Register / Login	Inputs, validation, errors	/auth/register + /auth/login
My appointments	Upcoming bookings, cancel button	/appointments/mine + cancel
Admin services	Add / edit / disable service	Admin service routes
Admin hours	Days and open/close time inputs	Admin hours route
Admin appointments	Daily list and filters	GET /admin/appointments

Suggested React state for booking

BookPage.jsx - key variables

```
const [serviceId, setServiceId] = useState('');
const [date, setDate] = useState(''); // e.g. 2026-11-10
const [slots, setSlots] = useState([]); // ISO UTC strings
const [selectedSlot, setSelectedSlot] = useState('');
const [loading, setLoading] = useState(false);
const [error, setError] = useState('');

// On date change, fetch /api/slots?date=...
// On confirm, POST { serviceId, startAt: selectedSlot }
// If response is 409, refetch slots and show a friendly message.
```

Basic UX polish that recruiters notice

- Disable Confirm while saving; avoid accidental double-click requests.
- Show a clear success state with service, date, and time.
- Show "No times available" instead of an empty broken grid.
- Keep labels clear and buttons easy to tap on a phone.

QUALITY / TEST IT

Test the cases that matter most

TEST	EXPECTED RESULT
Customer sends role=admin at register	Account still has customer role
Customer calls an admin route	403 Forbidden
Unauthenticated user books	401 Unauthorized
Customer books yesterday	400 Bad Request
Customer books 08:30 when opens at 09:00	400 Bad Request
Customer books 10:15 off the grid	400 Bad Request
Two users book 10:00 simultaneously	Exactly one 201, one 409
Customer cancels own future booking	It becomes available again
Customer tries to cancel someone else	No update; forbidden / not found
Inactive service is booked	Request rejected

The race-condition test idea

Illustrative integration-test pattern

```
const [a, b] = await Promise.all([
  bookAs(customerA, '2026-11-10T04:15:00.000Z'),
  bookAs(customerB, '2026-11-10T04:15:00.000Z')
]);

expect([a.status, b.status].sort()).toEqual([201, 409]);
```

The two user sessions must be separate. Run the test against a clean test database with the unique index built. **bookAs()** is a test helper you write.

> **ONE RULE FOR HIRING-QUALITY CODE**
>
> Do not only test the happy path. Interviewers care whether you thought about authorization, invalid input, duplicate requests, and failure messages.

DEPLOYMENT / PORTFOLIO

Deploy, document, and showcase it

Deployment checklist

- ☐ Put the MongoDB connection string and JWT secret in server environment variables.
- ☐ Deploy Express to a Node-compatible host; configure production HTTPS.
- ☐ Deploy React; set an API base URL and verify it points at the deployed backend.
- ☐ For cookie auth, configure same-site deployment or correct credentialed CORS, cookie attributes, and CSRF protection.
- ☐ Run migration/index setup and a seed script for your demo services + admin.
- ☐ Test login and booking on the deployed site, not only localhost.
- ☐ Check logs for errors and add a useful /api/health endpoint.

Your README should show

- A clear one-paragraph overview and screenshots or a short demo GIF.
- Technologies used, feature checklist, and routes for customer/admin.
- An architecture sketch and the four MongoDB models with relationships.
- Environment variable names (not actual secrets), setup commands, and demo links.
- A paragraph about how you prevented double booking with a unique partial index.

> RESUME BULLET EXAMPLE
>
> **Built SlotWise, a MERN appointment booking app** with customer/admin roles, UTC-based 30-minute scheduling, protected Express APIs, and MongoDB partial unique indexing to prevent double bookings. Deployed a live demo and added API tests for booking conflicts.

Only claim features and tests you actually finished. Replace placeholder values with your own project details.

FINAL PAGE / PRIORITIES AND LINKS

What to add later - and what to skip now

Best optional upgrades, in order

WHEN THE MVP WORKS	WHY IT HELPS
1. Rescheduling	Shows update rules and collision handling
2. Staff members	Introduces providerId and per-person slot indexes
3. Service durations	Requires safe overlap checks across multiple 30-min blocks
4. Date-specific closures	Holidays and admin blocked dates
5. Email reminders	Background delivery and notifications

YOUR FIRST SHIPPING MILESTONE

A customer can log in, book a 30-minute slot, see it on their profile, cancel it, and have the slot reopened. The admin can manage services and opening hours. A duplicate booking gets 409. **Ship this first.**

Official references

MongoDB - Partial indexes and uniqueness
https://www.mongodb.com/docs/manual/core/index-partial/

Mongoose - Validation and indexes
https://mongoosejs.com/docs/validation.html

Mongoose - Referenced documents
https://mongoosejs.com/docs/populate.html

Express - Routing and middleware
https://expressjs.com/en/guide/routing.html

MDN - UTC ISO timestamps
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toISOString

Luxon - Date and timezone handling
https://moment.github.io/luxon/

Prepared 27 October 2026. Code blocks are educational starting points, not a finished copy-and-run application. Complete middleware, validation, error handling, and environment setup are still required.